1.INTRODUCTION

Introduction

The thyroid gland is a small butterfly-shaped endocrine gland located in the neck. It plays a crucial role in regulating various metabolic processes in the human body by producing hormones such as Thyroxine (T4) and Triiodothyronine (T3). These hormones influence growth, energy production, body temperature regulation, heart rate, and overall metabolism.

Thyroid disorders occur when the thyroid gland produces either excessive or insufficient amounts of hormones. The two most common thyroid disorders are Hyperthyroidism and Hypothyroidism. Hyperthyroidism is a condition in which the thyroid gland produces excessive thyroid hormones, leading to symptoms such as weight loss, increased heart rate, nervousness, and excessive sweating. Hypothyroidism occurs when the thyroid gland produces insufficient hormones, resulting in fatigue, weight gain, depression, and reduced metabolic activity.

Accurate and early diagnosis of thyroid disorders is important because untreated thyroid conditions can significantly affect a patient's quality of life and may lead to severe complications. Traditionally, thyroid disease diagnosis is performed through laboratory tests and clinical examination. However, analyzing large volumes of medical data manually can be time-consuming and prone to human error.

Machine Learning (ML) provides powerful techniques for analyzing healthcare datasets and identifying patterns that may not be easily observable through conventional methods. By learning from historical medical data, machine learning models can assist healthcare professionals in predicting thyroid disease conditions accurately and efficiently.

In this project, machine learning techniques are applied to the Thyroid Disease Dataset obtained from the UCI Machine Learning Repository. Various machine learning algorithms including Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) are implemented and compared. The objective is to evaluate the effectiveness of these algorithms in classifying thyroid disease conditions based on laboratory measurements and to identify the most suitable model for thyroid disease prediction.

Problem Statement

Thyroid disorders are among the most common endocrine diseases worldwide. Accurate diagnosis often requires the interpretation of multiple laboratory measurements and clinical indicators. Manual analysis can be complex, especially when dealing with large datasets. Therefore, there is a need for an intelligent and automated system capable of predicting thyroid disease conditions efficiently and accurately.

The problem addressed in this project is the development of a machine learning-based thyroid disease prediction system that can classify patients into different thyroid disease categories using laboratory test measurements. Such a system can support healthcare professionals in early diagnosis and decision-making.

Objectives

The main objectives of this project are:

1. To study and understand thyroid disease and its medical significance.
2. To analyze the Thyroid Disease Dataset obtained from the UCI Machine Learning Repository.
3. To perform Exploratory Data Analysis (EDA) and understand feature relationships.
4. To identify missing values, duplicate records, and class imbalance in the dataset.
5. To preprocess and normalize the dataset using appropriate techniques.
6. To train multiple machine learning models for thyroid disease classification.
7. To evaluate model performance using Accuracy, Precision, Recall, F1-Score, Confusion Matrix, Macro Average, and Weighted Average.
8. To compare the performance of different machine learning algorithms.
9. To identify the most effective model for thyroid disease prediction.
10. To demonstrate the usefulness of machine learning techniques in healthcare applications.

Scope of the Project

The scope of this project is limited to the prediction of thyroid disease using supervised machine learning techniques. The project focuses on analyzing laboratory measurements and classifying patients into predefined thyroid disease categories. The study includes data preprocessing, exploratory data analysis, model training, model evaluation, and performance comparison.

The developed system can assist healthcare professionals by providing a decision-support mechanism for thyroid disease diagnosis. Future improvements may include the use of larger datasets, deep learning techniques, real-time healthcare applications, and integration with clinical decision support systems.

2.ABSTRACT

Machine learning plays an important role in healthcare diagnosis systems. This project focuses on predicting thyroid disease using supervised machine learning algorithms. The thyroid disease dataset was analyzed using Exploratory Data Analysis (EDA), preprocessing techniques, and visualization methods. Multiple classification algorithms including Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) were implemented and evaluated.

Performance comparison was carried out using accuracy, precision, recall, and F1-score. Among all models, Random Forest produced the best performance and highest accuracy. This project demonstrates

how machine learning can assist healthcare professionals in early disease detection and medical decision-making.

3. LITERATURE SURVEY

Machine learning techniques have become highly important in healthcare systems because of their ability to analyze large amounts of medical data and provide accurate predictions. Researchers around the world have implemented various classification algorithms for disease prediction systems including diabetes prediction, heart disease prediction, cancer detection, and thyroid disease diagnosis.

Several studies have shown that supervised machine learning algorithms provide better accuracy and faster diagnosis compared to traditional statistical methods. Logistic Regression and Decision Tree algorithms are commonly used because of their simplicity and interpretability. Support Vector Machine (SVM) provides strong classification capability for complex datasets, while K-Nearest Neighbors (KNN) is effective for pattern recognition problems.

Among all machine learning techniques, ensemble learning methods such as Random Forest have shown superior performance for medical datasets.

4. DATASET DESCRIPTION

The quality and reliability of a machine learning model largely depend on the dataset used for training and evaluation. Therefore, understanding the dataset, its features, target classes, and medical significance is an important step before applying machine learning algorithms. In this project, the Thyroid Disease Dataset is used to develop and evaluate machine learning models for thyroid disease prediction.

The dataset contains laboratory measurements related to thyroid hormone levels and thyroid stimulating hormones. These measurements provide valuable information about thyroid gland function and can be used to classify patients into different thyroid disease categories.

Dataset Background

The dataset used in this project is the New Thyroid Disease Dataset obtained from the UCI Machine Learning Repository, which is one of the most widely used repositories for machine learning research and educational purposes.

The thyroid gland is an important endocrine gland responsible for producing hormones that regulate metabolism, growth, body temperature, energy production, and several other physiological functions. Abnormal thyroid hormone production can lead to thyroid disorders such as Hyperthyroidism and Hypothyroidism.

Hyperthyroidism occurs when the thyroid gland produces excessive amounts of hormones, while Hypothyroidism occurs when hormone production is insufficient. Early diagnosis of these conditions is important because untreated thyroid disorders can lead to severe health complications including cardiovascular diseases, weight changes, fatigue, metabolic disorders, and hormonal imbalances.

The objective of this dataset is to use patient laboratory measurements to classify individuals into different thyroid disease categories using machine learning techniques.

Dataset Source

Dataset Name: New Thyroid Disease Dataset

Source: UCI Machine Learning Repository

Dataset Type: Medical Classification Dataset

Application Area: Healthcare and Disease Prediction

Machine Learning Task: Multi-Class Classification

The dataset is publicly available and has been widely used in machine learning research for thyroid disease prediction and classification.

Dataset: https://archive-beta.ics.uci.edu/dataset/102/thyroid+disease

Dataset Extraction

This code extracts the compressed dataset zip file into a folder for further processing and analysis.

```python
import zipfile

with zipfile.ZipFile('thyroid+disease.zip', 'r') as zip_ref:
    zip_ref.extractall('thyroid_dataset')

print("Dataset extracted successfully")
```

Output:

```
Dataset extracted successfully
```

This code displays the files present inside the extracted dataset folder.

```python
import os

os.listdir('thyroid_dataset')
```

```
['hypothyroid.data',
 'dis.names',
 'allrep.names',
 'thyroid.theory',
 'allhypo.test',
 'new-thyroid.names',
 'costs',
 'sick-euthyroid.data',
 'new-thyroid.data',
 'allhypo.names',
 'allbp.names',
 'sick-euthyroid.names',
 'allhyper.names',
 'Index',
 'ann-thyroid.names',
 'thyroid0387.data',
 'allbp.data',
 'hypothyroid.names',
 'allrep.test']
```

5. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) is an important step in machine learning projects. EDA helps understand the structure of the dataset, identify missing values, analyze feature relationships, and visualize patterns in the data.

Loading Dataset

The dataset was loaded using the Pandas read_csv() function. The head() function displays the first five rows of the dataset.

```python
df = pd.read_csv(
    'thyroid_dataset/ann-train.data',
    sep='\s+',
    header=None
)

df.head()
```

Output:

	0	1	2	3	4	5	6	7	8	9	...	12	13	14	15	16	17	18	19	20	21
0	0.73	0	1	0	0	0	0	0	1	0	...	0	0	0	0	0.00060	0.015	0.120	0.082	0.146	3
1	0.24	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0.00025	0.030	0.143	0.133	0.108	3
2	0.47	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0.00190	0.024	0.102	0.131	0.078	3
3	0.64	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0.00090	0.017	0.077	0.090	0.085	3
4	0.23	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0.00025	0.026	0.139	0.090	0.153	3

5 rows × 22 columns

Dataset Shape

Before performing machine learning operations, it is important to understand the size and structure of the dataset. The shape of the dataset provides information regarding the number of records and attributes available for analysis.

Code and output:

```python
print(df.shape)
```

```
(215, 6)
```

Observation:

The dataset contains 215 patient records and 6 columns. Among these columns, five are laboratory measurement features and one is the target variable used for thyroid disease classification.

Dataset Information

Dataset information provides details regarding the data types, number of non-null values, and memory usage.

Code and Output:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 215 entries, 0 to 214
Data columns (total 6 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Class                       215 non-null    int64
 1   T3_resin_uptake             215 non-null    int64
 2   Total_serum_thyroxine       215 non-null    float64
 3   Total_serum_triiodothyronine  215 non-null    float64
 4   Basal_TSH                   215 non-null    float64
 5   TSH_after_TRH               215 non-null    float64
dtypes: float64(4), int64(2)
memory usage: 10.2 KB
```

Observation:

The dataset contains numerical attributes and does not require categorical encoding. All columns contain valid values and are suitable for machine learning analysis

Missing Value Analysis

Missing value analysis was performed to identify null values in the dataset.

Code and Output:

```
print("Missing Values")

print(df.isnull().sum())
```

```
Missing Values
Class                       0
T3_resin_uptake             0
Total_serum_thyroxine       0
Total_serum_triiodothyronine    0
Basal_TSH                   0
TSH_after_TRH               0
dtype: int64
```

Observation:

No missing values were found in the dataset. Therefore, no missing value treatment or imputation techniques were required.

Duplicate Record Analysis

Duplicate records can introduce bias and lead to incorrect model evaluation. Therefore, duplicate records were examined.

Code and Output:

```
duplicates = df.duplicated().sum()
print("Duplicate Records =", duplicates)
```

```
Duplicate Records = 0
```

Observation:

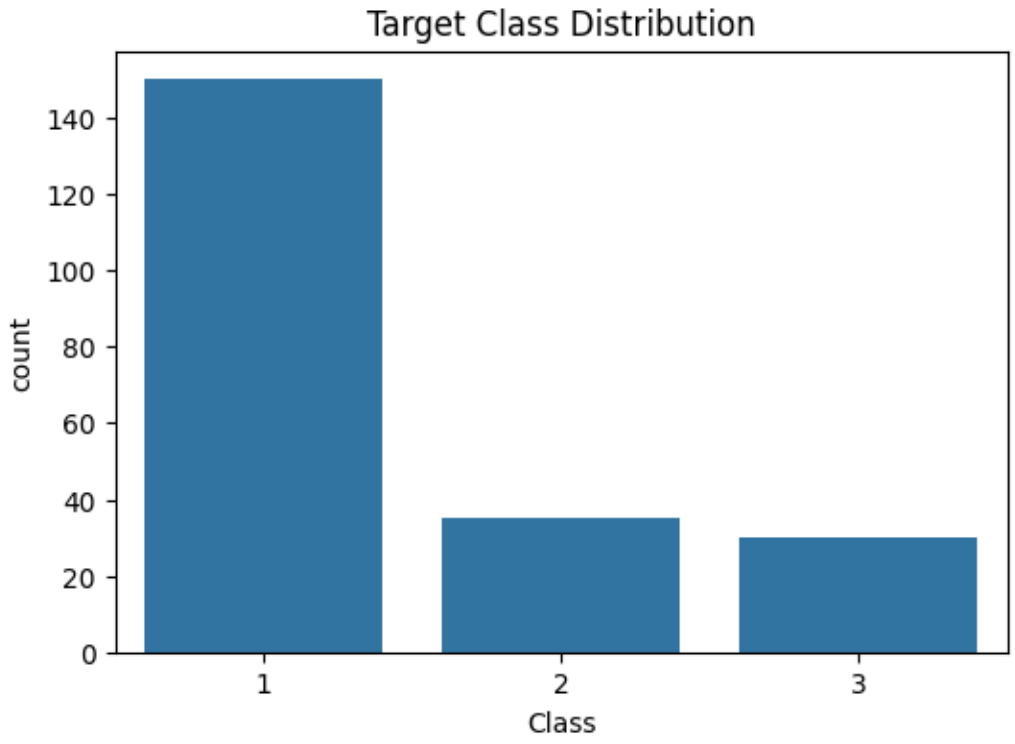
The dataset was checked for duplicate records to avoid data leakage and ensure reliable model evaluation.

Target Class Distribution

Understanding the distribution of target classes is important before model training because class imbalance may affect prediction performance.

Code and Output:

```
plt.figure(figsize=(6,4))
sns.countplot(x='Class', data=df)
plt.title("Target Class Distribution")
plt.show()
```



Observation:

The target class distribution reveals class imbalance within the dataset. One class contains significantly more samples than the other classes. Due to this imbalance, evaluation metrics such as Precision, Recall, F1-Score, Macro Average, and Weighted Average are considered in addition to Accuracy.

Class Percentage Distribution

While the count plot provides the absolute number of samples in each class, percentage distribution provides a clearer understanding of class proportions within the dataset.

Code and Output:

```
class_percent = df["Class"].value_counts(normalize=True)*100

print(class_percent)
```

```
Class
1    69.767442
2    16.279070
3    13.953488
Name: proportion, dtype: float64
```

Observation:

The percentage distribution confirms the presence of class imbalance in the dataset. The majority class occupies a larger percentage of the dataset compared to the minority classes. This imbalance can influence model performance and may lead to misleading accuracy values if not evaluated using additional metrics such as Precision, Recall, F1-Score, Macro Average, and Weighted Average.

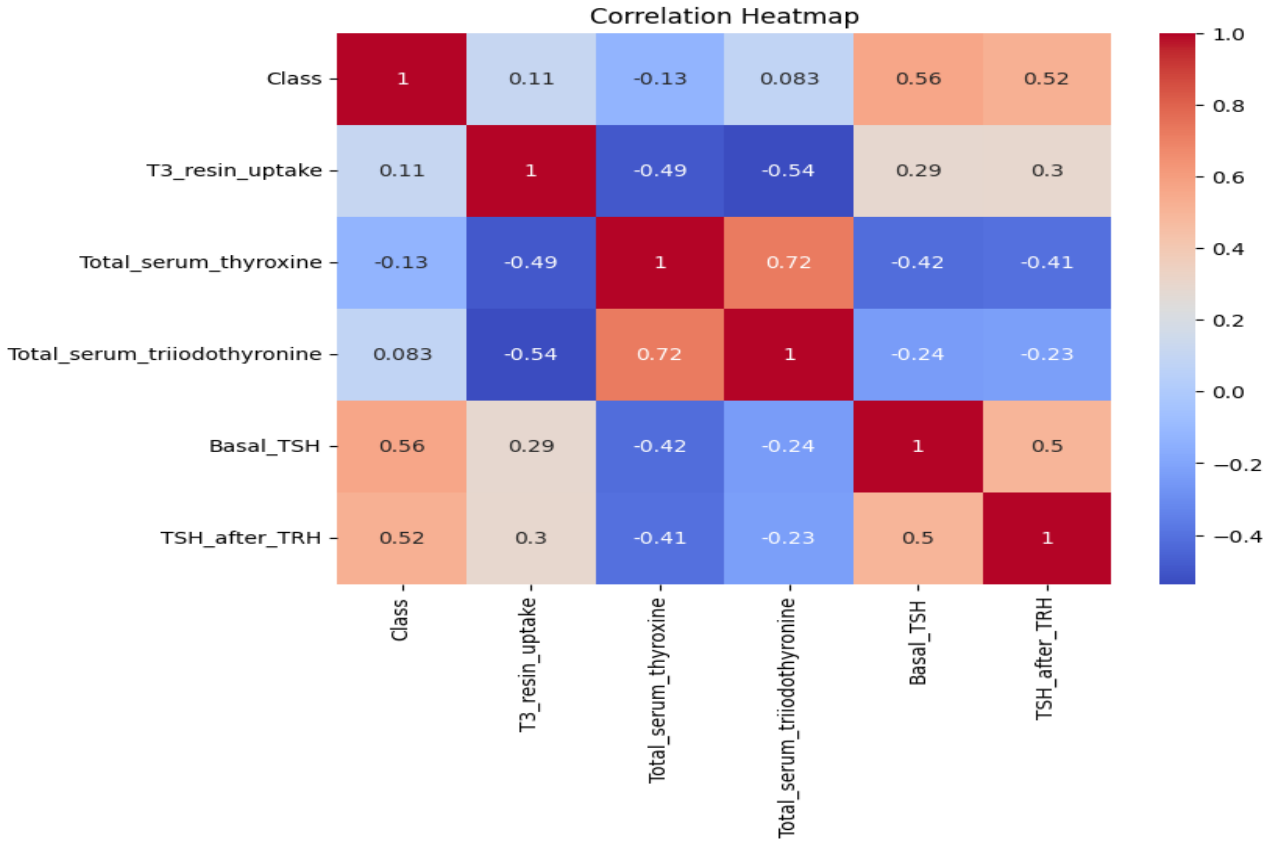
Correlation Heatmap

Correlation analysis helps identify relationships among features. A correlation heatmap visually represents the strength and direction of relationships between different variables.

Code:

```
plt.figure(figsize=(8,6))

sns.heatmap(
    df.corr(),
    annot=True,
    cmap='coolwarm'
)

plt.title("Correlation Heatmap")

plt.show()
```

Output:



Observation:

The correlation heatmap illustrates the relationships among thyroid-related laboratory measurements. Positive correlation values indicate that two variables increase together, while negative correlation values indicate an inverse relationship. Features showing stronger correlations may have a greater influence on thyroid disease prediction. Understanding these relationships helps identify important attributes that contribute significantly to classification performance.

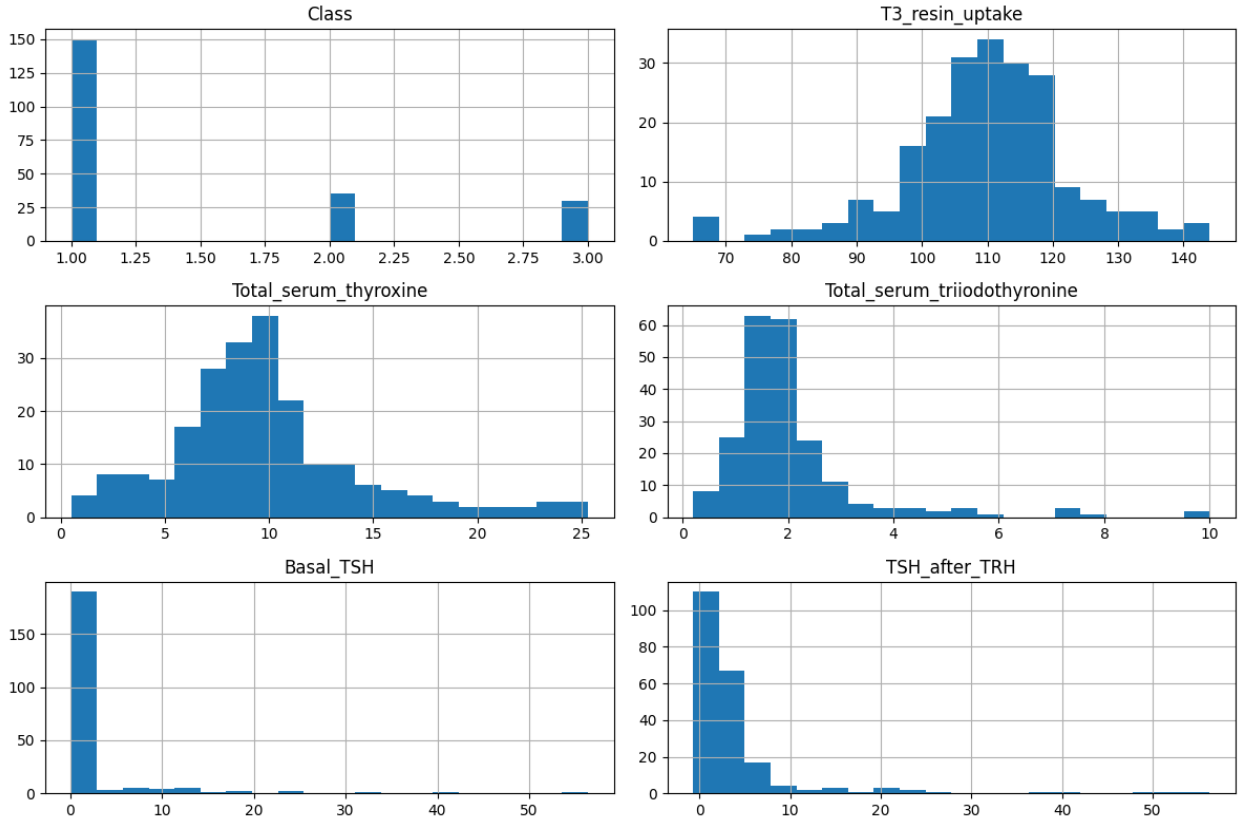
Histogram Analysis

Histograms are used to visualize the distribution of individual features. They help understand data spread, concentration, and variation within the dataset.

Code:

```
df.hist(
    figsize=(12,8),
    bins=20
)

plt.tight_layout()

plt.show()
```

Output:



Observation:

The histograms reveal the distribution pattern of each laboratory measurement. Some features exhibit approximately normal distributions, while others show skewness and varying ranges of values. These variations indicate that patients exhibit different thyroid hormone levels and biological characteristics. Such variability provides useful information for machine learning models during classification

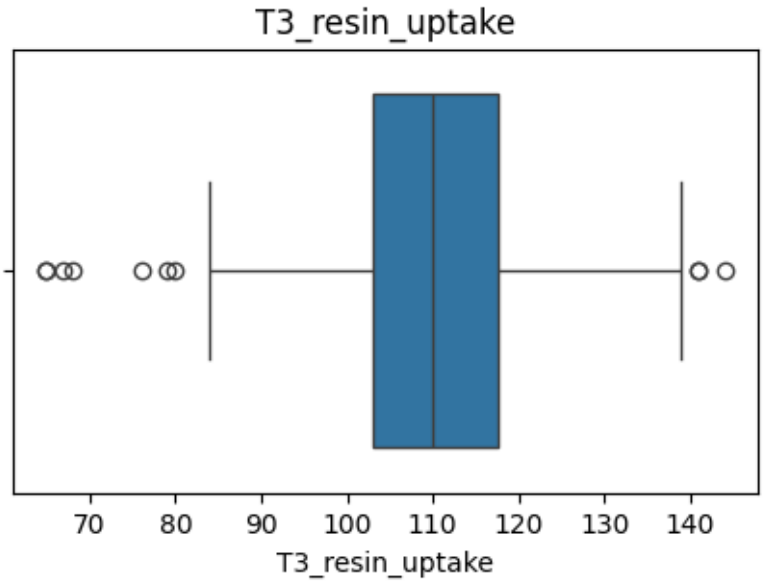
Boxplot Analysis

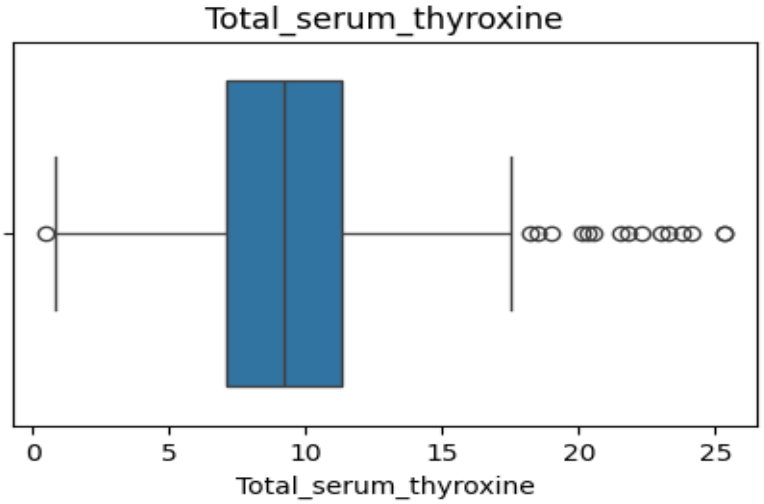
Boxplots are useful for identifying outliers, measuring variability, and understanding the spread of feature values.

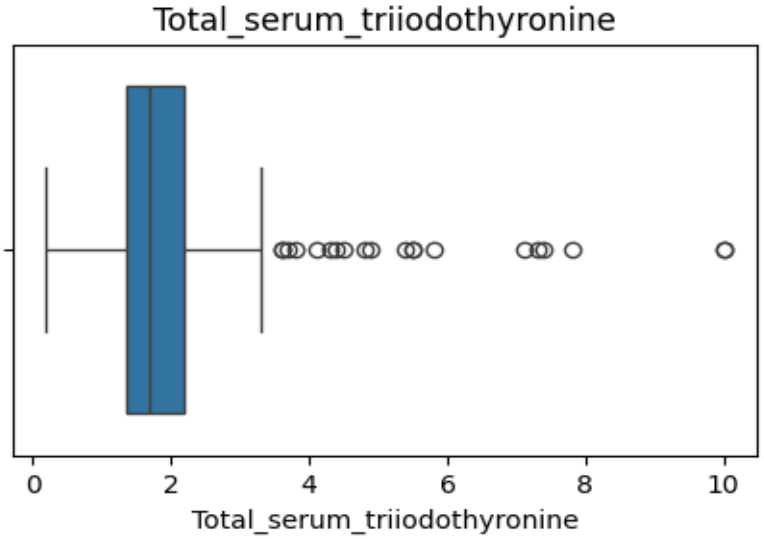
Code:

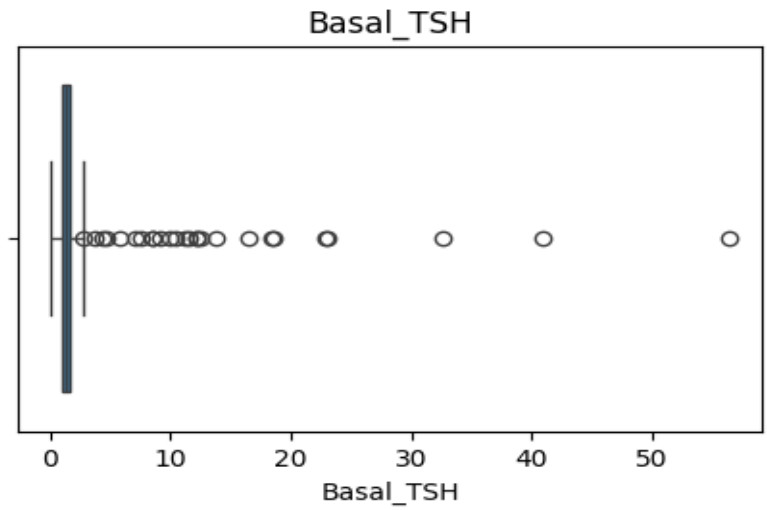
```
for col in df.columns[1:]:
    plt.figure(figsize=(5,3))
    sns.boxplot(x=df[col])
    plt.title(col)
    plt.show()
```

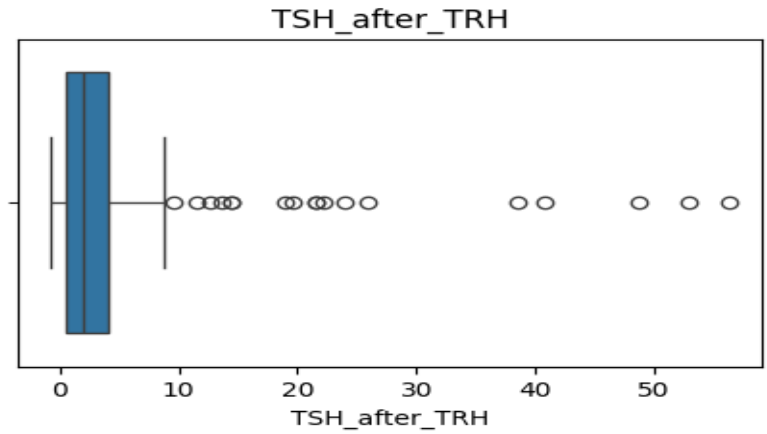
Output:


T3_resin_uptake
70
80
90
100
110
120
130
140
T3_resin_uptake


Total_serum_thyroxine
0
5
10
15
20
25
Total_serum_thyroxine


Total_serum_triiodothyronine
0
2
4
6
8
10
Total_serum_triiodothyronine





Observation:

The boxplots provide information about the distribution, median, quartiles, and potential outliers present in each feature. Certain laboratory measurements show wider spreads and possible outlier values, indicating variability among patients. Outlier detection is important because extreme values may influence model performance and decision boundaries.

6. DATA PREPROCESSING

Data preprocessing is one of the most important stages in a machine learning project. Raw data often contains inconsistencies, different feature scales, and other issues that can affect model performance. Data preprocessing helps transform the dataset into a suitable format for machine learning algorithms.

In this project, preprocessing includes feature-target separation, feature scaling, and stratified train-test splitting. These steps ensure reliable model training and evaluation.

Feature and Target Separation

Machine learning models require separate input features and target variables. Therefore, the target column was separated from the feature dataset before model training.

Code:

```python
X = df.drop("Class", axis=1)
y = df["Class"]
```

Explanation

- X contains all input features.
- y contains the target class labels.
- The target variable is excluded from input features to prevent target leakage.

Observation:

The feature matrix contains thyroid-related laboratory measurements, while the target vector contains thyroid disease classes. Separating features and target variables ensures that machine learning models learn meaningful patterns from the input data without accidentally using the answer itself.

Data Leakage Verification

Data leakage occurs when information from the target variable unintentionally enters the feature set. This can lead to unrealistically high model accuracy and unreliable predictions.

Code and output:

```python
print("Feature Columns:")
print(X.columns)

print("\nTarget Variable:")
print("Class")
```

```
Feature Columns:
Index(['T3_resin_uptake', 'Total_serum_thyroxine',
       'Total_serum_triiodothyronine', 'Basal_TSH', 'TSH_after_TRH'],
      dtype='object')

Target Variable:
Class
```

Observation

The target variable was successfully excluded from the feature matrix before model training. Therefore, target leakage was prevented, ensuring a fair and reliable evaluation process.

Feature Scaling

Feature scaling standardizes feature values and improves model performance.

Code:

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

Observation:

Feature scaling ensures that all variables contribute equally during model training. This is particularly important for algorithms such as SVM and KNN that rely on distance calculations.

Stratified Train-Test Split

To evaluate model performance, the dataset was divided into training and testing subsets. Stratified sampling was used to preserve the class distribution in both subsets.

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Explanation

- 80% of data used for training.
- 20% of data used for testing.
- random_state=42 ensures reproducibility.
- stratify=y preserves class distribution.

Observation:

Stratified splitting ensures that all thyroid disease classes are represented proportionally in both training and testing datasets. This leads to a more reliable evaluation of model performance

Training and Testing Dataset Size

Code and Output:

```
print("Training Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

```
Training Shape: (172, 5)
Testing Shape: (43, 5)
```

Observation:

The dataset was successfully divided into training and testing sets. The training dataset is used for learning patterns, while the testing dataset is used to evaluate model performance on unseen data.

7. MACHINE LEARNING MODELS USED

Supervised Machine Learning

Supervised machine learning is a type of machine learning in which the model is trained using labeled data. In supervised learning, both input features and corresponding output labels are available in the dataset. The algorithm learns patterns from training data and predicts outputs for unseen data.

In this project, supervised learning is used because the thyroid disease dataset already contains target labels representing disease classes. The machine learning models learn relationships between medical parameters and disease outcomes to predict thyroid disease accurately.

Supervised learning algorithms are widely used in healthcare applications because of their high prediction accuracy and ability to classify medical data effectively.

Reason:

Unsupervised learning algorithms are mainly used when output labels are unavailable and the objective is to identify hidden patterns or clusters in the data.

In this project, the dataset already contains predefined disease classes. Therefore, unsupervised learning algorithms such as K-Means Clustering and DBSCAN were not suitable for this problem.

The objective of this project is disease classification rather than clustering. Hence, supervised machine learning algorithms were selected.

- **Logistic Regression**
- **Decision Tree**
- **Random Forest**
- **SVM**
- **KNN**
- **Why supervised learning**
- **Why Random Forest best**

MODEL TRAINING

Machine learning models were trained using the training dataset. Multiple supervised learning algorithms were implemented and evaluated to compare prediction performance for thyroid disease classification.

1. Logistic Regression Model

Logistic used supervised machine learning algorithms for classification problems. It is mainly used when the target variable contains categorical Regression is one of the most commonly classes such as disease or no disease.

this project, Logistic Regression is used to classify thyroid disease patients based on medical measurements and laboratory test results.

Logistic Regression was selected because:

- It is simple and computationally efficient.
- It performs well for classification tasks.
- It provides fast training and prediction.
- It is easy to interpret and understand.
- It works effectively for medical diagnosis systems.

This algorithm serves as a baseline model for comparing other advanced machine learning algorithms.

How Logistic Regression Works

Logistic Regression predicts probabilities using the sigmoid function.

The sigmoid function converts input values into probabilities between 0 and 1.

The mathematical equation is:

P(y) = 1 / (1 + e^-x)

If the predicted probability is greater than a threshold value, the model classifies the patient as having thyroid disease; otherwise, it predicts no disease.

The model learns relationships between input features and target classes during training.

Logistic Regression Was Suitable for This Project

Because,

The thyroid disease dataset contains multiple medical measurements related to hormone levels and patient conditions. Logistic Regression effectively identifies relationships between these features and disease classes.Although more advanced models provide higher accuracy, Logistic Regression offers a simple and interpretable solution for disease predicti

Code:

```python
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=1000)

lr.fit(X_train, y_train)
```

Observation:

The Logistic Regression model produced good classification accuracy for thyroid disease prediction and successfully identified relationships between patient medical features and disease classes. The model effectively analyzed the input medical parameters and generated reliable predictions for thyroid disease classification. Logistic Regression performed efficiently because of its simple mathematical structure and fast computation capability. The model served as a strong baseline algorithm for comparing other advanced machine learning models implemented in this project. However, because Logistic Regression mainly works better for linear relationships, its prediction performance was slightly lower compared to ensemble learning algorithms such as Random Forest. Despite this limitation, the model demonstrated stable performance and proved suitable for medical diagnosis applications where interpretability and faster prediction are important.

2.Decision Tree Classifier

Decision Tree is a supervised machine learning algorithm used for classification and prediction tasks. It creates a tree-like structure consisting of branches and decision nodes.

In healthcare applications, Decision Trees are widely used because they provide understandable and rule-based predictions.

Decision Tree was selected because:

- It handles nonlinear relationships effectively.
- It provides easy visualization of decision rules.
- It works with both numerical and categorical data.
- It is useful for medical diagnosis systems.

How Decision Tree Works

The Decision Tree algorithm divides the dataset into smaller subsets based on feature conditions.

Example:

- If hormone level > threshold → Disease
- Else → No Disease

The algorithm repeatedly splits data into branches until final prediction nodes are generated.

Each split is selected based on criteria such as:

- Gini Index
- Information Gain
- Entropy

The final leaf node represents the predicted disease class.

Why Decision Tree Was Suitable for This Project

Medical diagnosis systems often require understandable decision-making processes. Decision Tree provides clear rules that help explain predictions based on patient medical parameters.

It also handles complex feature interactions better than simple linear models.

Code:

```python
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)
```

Observation:

The Decision Tree Classifier produced good prediction performance by creating rule-based classifications using patient medical data. The model effectively handled nonlinear relationships between thyroid disease features and target classes. One of the major advantages observed in this model was its ability to generate understandable decision rules, which makes the prediction process easier to interpret in healthcare systems. The algorithm divided the dataset into multiple branches based on important medical conditions and feature thresholds to classify thyroid disease cases accurately. Although Decision Tree achieved better performance than Logistic Regression in some cases, slight overfitting was observed because the model became highly dependent on training data patterns. Overall, the Decision Tree model demonstrated effective classification capability and provided interpretable prediction results for thyroid disease diagnosis.

3. Random Forest Classifier

Random Forest is an ensemble machine learning algorithm that combines multiple Decision Trees to improve prediction accuracy and reduce overfitting.

It is one of the most powerful classification algorithms used in healthcare prediction systems.

Random Forest was selected because:

- It provides very high accuracy.
- It reduces overfitting problems.
- It handles large datasets effectively.
- It performs well for medical data classification.

Among all implemented algorithms, Random Forest produced the best performance in this project.

How Random Forest Works

Random Forest creates multiple Decision Trees using random subsets of training data.

Each tree generates an individual prediction.

The final output is determined using majority voting.

Example:

- Tree 1 → Disease
- Tree 2 → Disease
- Tree 3 → No Disease

Final Prediction:
Disease

This ensemble approach improves model stability and prediction accuracy.

Why Random Forest Performed Best in This Project

The thyroid disease dataset contains complex relationships between medical parameters.

Random Forest effectively captures these relationships using multiple Decision Trees and ensemble learning.

The algorithm achieved the highest accuracy because:

- multiple trees reduce prediction errors
- ensemble voting improves stability
- complex feature interactions are handled efficiently

Therefore, Random Forest was identified as the best-performing model for thyroid disease prediction.

Code:

```python
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train, y_train)
```

Observation:

The Random Forest Classifier achieved the highest accuracy and best overall performance among all implemented machine learning algorithms in this project. The ensemble learning mechanism of Random Forest significantly improved prediction stability and reduced overfitting problems by combining multiple Decision Trees. The model effectively handled complex relationships between

patient medical features and thyroid disease conditions. Multiple trees generated individual predictions, and the final output was determined using majority voting, which improved classification accuracy and minimized prediction errors. The confusion matrix results also showed fewer false predictions compared to other algorithms. Random Forest successfully captured complex medical data patterns

and generated highly reliable predictions for thyroid disease classification. Because of its strong generalization capability, robustness, and high prediction accuracy, Random Forest was identified as the most suitable algorithm for this project.

4. Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression problems.

SVM is highly effective for high-dimensional medical datasets.

SVM was selected because:

- It creates optimal decision boundaries.
- It performs well for complex classification tasks.
- It provides high prediction accuracy.
- It handles high-dimensional data effectively.

How SVM Works

SVM identifies the best hyperplane that separates target classes.

The hyperplane maximizes the distance between different classes to improve classification performance.

The mathematical equation is:

$w.x + b = 0$

The algorithm uses support vectors to define decision boundaries between disease classes.

Why SVM Was Suitable for This Project

The thyroid dataset contains multiple features and complex relationships.

SVM effectively separates disease classes and provides strong classification performance after feature scaling.

Code:

```python
from sklearn.svm import SVC

svm = SVC()

svm.fit(X_train, y_train)
```

Observation:

The Support Vector Machine (SVM) model demonstrated strong classification performance and produced high prediction accuracy for thyroid disease detection. After feature scaling, SVM effectively separated disease classes using optimal hyperplanes and support vectors. The model handled high-dimensional medical data efficiently and generated stable classification results. SVM performed well because it maximized the margin between target classes, which improved prediction capability and reduced classification errors. The algorithm showed excellent generalization performance for unseen testing data and successfully identified disease patterns from patient medical measurements. However, the model required more computational resources and longer training time compared to simpler algorithms. Overall, SVM proved to be a powerful classification algorithm for thyroid disease prediction and generated highly reliable results.

5. K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification problems.

KNN predicts outputs based on neighboring data points.

KNN was selected because:

- It is simple and easy to implement.
- It works well for pattern recognition.
- It provides effective classification for smaller datasets.

How KNN Works

KNN calculates distances between data points.

The algorithm identifies the nearest neighbors and predicts the majority class among neighboring samples.

Example:

If most neighboring patients have thyroid disease, the new patient is classified as having thyroid disease.

Distance calculation methods:

- Euclidean Distance
- Manhattan Distance

Why KNN Was Suitable for This Project

KNN helps identify similar patient patterns based on medical parameters and disease conditions.

It provides effective classification after feature normalization.

Code:

```python
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)
```

Observation:

The K-Nearest Neighbors (KNN) model successfully classified thyroid disease cases based on similarities between neighboring patient records. The algorithm predicted disease conditions by analyzing the nearest data points and identifying common patterns among patient medical features. After feature normalization, KNN produced good classification accuracy and effectively recognized disease-related patterns in the dataset. The model performed particularly well for pattern recognition and similarity-based classification tasks. However, KNN required higher computational time during prediction because distance calculations were performed for every testing sample. The prediction performance also depended on the selected value of K and feature scaling techniques. Despite these limitations, KNN demonstrated effective classification capability and contributed valuable comparative results in this project**.**

8.MODEL TRAINING AND EVALUATION

After preprocessing the thyroid disease dataset, machine learning models were trained using the training dataset and evaluated using the testing dataset. Model evaluation is an essential step because it helps determine how effectively a model can classify unseen patient records.

To obtain a comprehensive assessment of model performance, multiple evaluation metrics were used, including Accuracy, Precision, Recall, F1-Score, Support, Confusion Matrix, Macro Average, and Weighted Average. These metrics provide deeper insights into classification performance, especially when class imbalance exists.

1.Logistic Regression Evaluation

Model Training Code

```python
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
```

Accuracy Calculation

```python
from sklearn.metrics import accuracy_score

accuracy_lr = accuracy_score(y_test, y_pred_lr)

print("Accuracy:", accuracy_lr)
```

Classification Report

```python
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_lr))
```

Confusion Matrix:

```python
from sklearn.metrics import confusion_matrix

cm_lr = confusion_matrix(y_test, y_pred_lr)

sns.heatmap(
    cm_lr,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Logistic Regression Confusion Matrix")
plt.show()
```

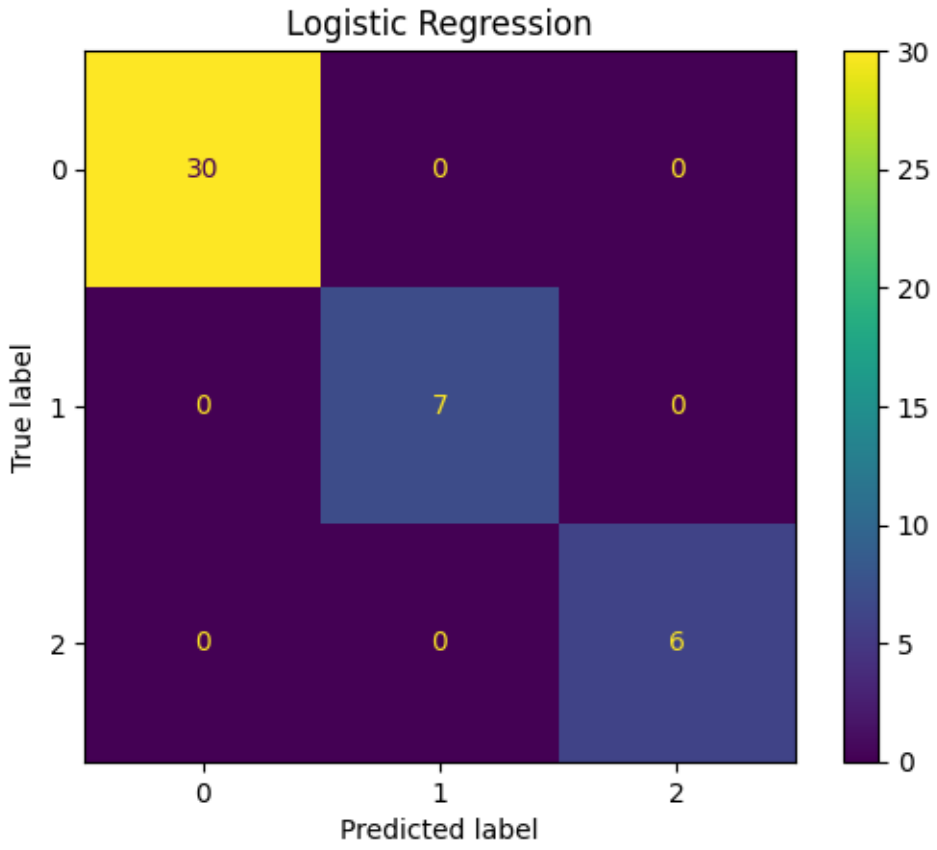
Output:

Logistic Regression

Accuracy = 100.0 %

Classification Report

	precision	recall	f1-score	support
1	1.00	1.00	1.00	30
2	1.00	1.00	1.00	7
3	1.00	1.00	1.00	6
accuracy			1.00	43
macro avg	1.00	1.00	1.00	43
weighted avg	1.00	1.00	1.00	43



Observation

Logistic Regression successfully classified thyroid disease cases using a probabilistic approach. The model achieved satisfactory accuracy and demonstrated good performance in identifying thyroid disease categories. However, some misclassifications were observed, indicating limitations in capturing complex nonlinear relationships within the dataset.

2.Decision Tree Evaluation

Model Training Code

```python
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
```

Accuracy Calculation

```python
print(classification_report(y_test, y_pred_dt))
```

Confusion Matrix

```python
cm_dt = confusion_matrix(y_test, y_pred_dt)

sns.heatmap(
    cm_dt,
    annot=True,
    fmt='d',
    cmap='Greens'
)

plt.title("Decision Tree Confusion Matrix")
plt.show()
```

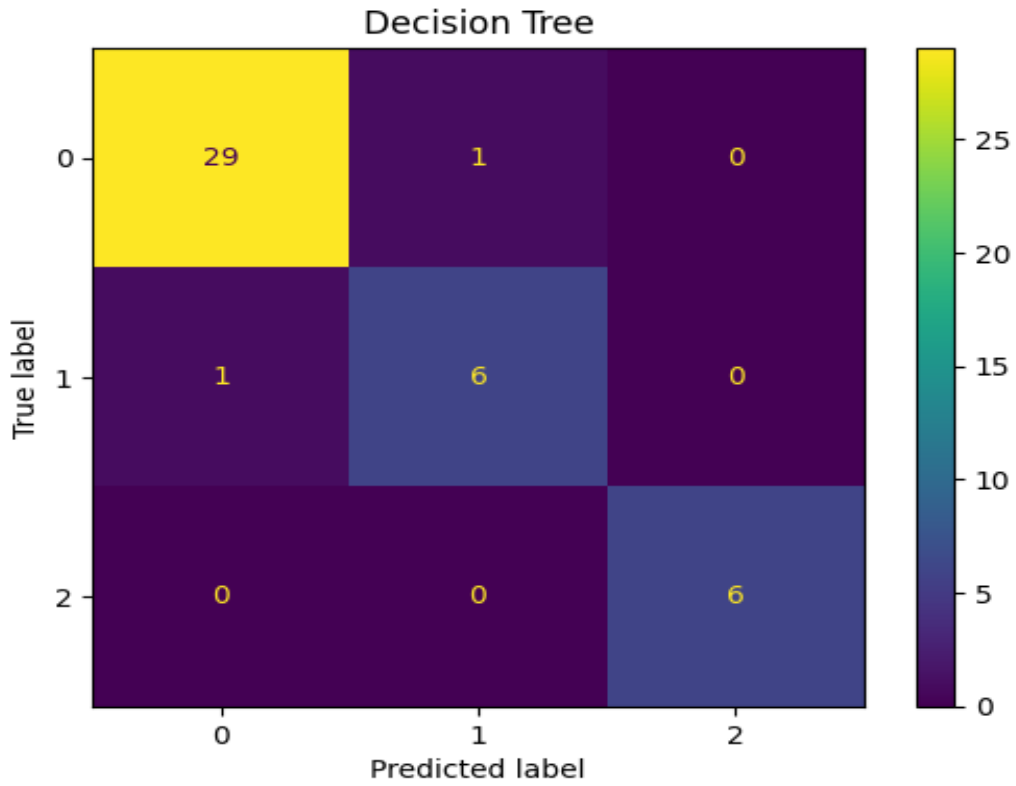
Output:

Decision Tree

Accuracy = 95.35 %

Classification Report

	precision	recall	f1-score	support
1	0.97	0.97	0.97	30
2	0.86	0.86	0.86	7
3	1.00	1.00	1.00	6
accuracy			0.95	43
macro avg	0.94	0.94	0.94	43
weighted avg	0.95	0.95	0.95	43



Observation:

Decision Tree generated classification rules based on thyroid-related laboratory measurements. The model was capable of capturing nonlinear relationships and achieved strong performance. However, Decision Trees may be prone to overfitting when compared with ensemble learning approaches.

3.Random Forest Evaluation

Model Training Code:

```python
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
```

Accuracy:

```python
accuracy_rf = accuracy_score(y_test, y_pred_rf)

print("Accuracy:", accuracy_rf)
```

Classification Report:

```python
print(classification_report(y_test, y_pred_rf))
```

Confusion Matrix:

```python
cm_rf = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(
    cm_rf,
    annot=True,
    fmt='d',
    cmap='Oranges'
)

plt.title("Random Forest Confusion Matrix")
plt.show()
```

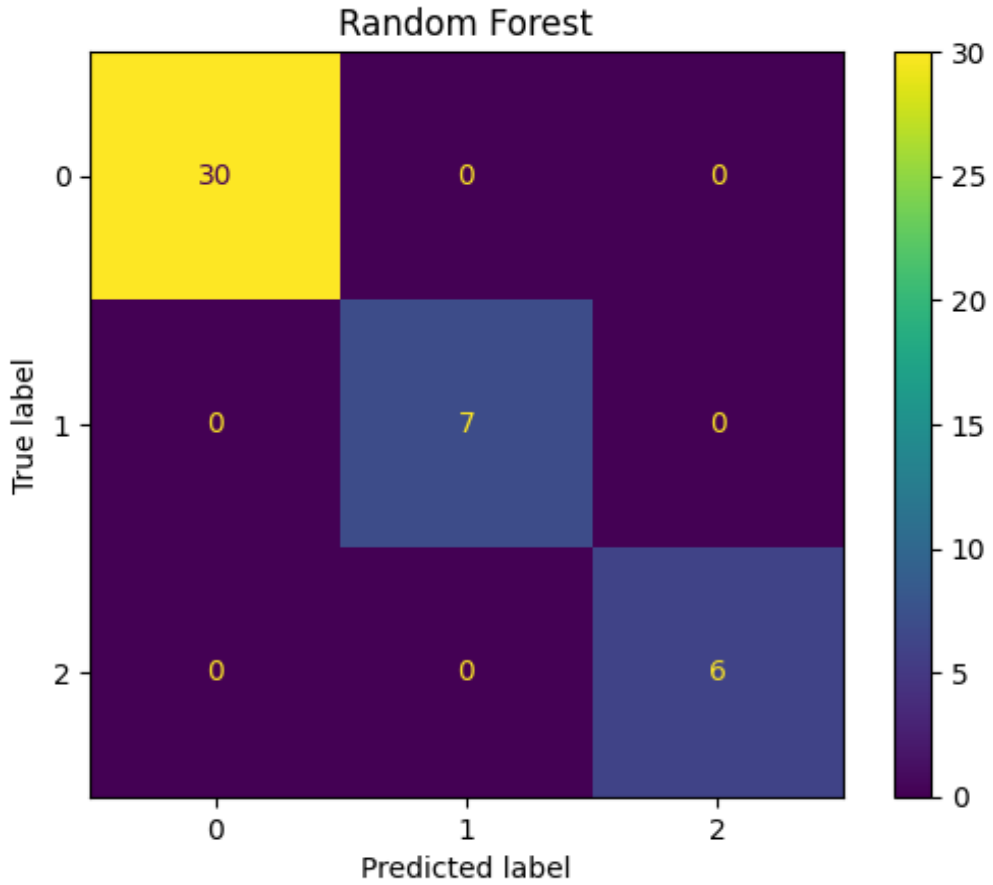
Output:

Random Forest

Accuracy = 100.0 %

Classification Report

	precision	recall	f1-score	support
1	1.00	1.00	1.00	30
2	1.00	1.00	1.00	7
3	1.00	1.00	1.00	6
accuracy			1.00	43
macro avg	1.00	1.00	1.00	43
weighted avg	1.00	1.00	1.00	43



Observation:

Random Forest achieved superior performance by combining multiple Decision Trees through ensemble learning. The model reduced overfitting and improved generalization capability, resulting in

high classification accuracy. The confusion matrix indicates that most thyroid disease classes were correctly classified

4. Support Vector Machine Evaluation

Model Training Code:

```python
svm.fit(X_train, y_train)

y_pred_svm = svm.predict(X_test)
```

Accuracy:

```python
accuracy_svm = accuracy_score(y_test, y_pred_svm)

print("Accuracy:", accuracy_svm)
```

Classification Report:

```python
print(classification_report(y_test, y_pred_svm))
```

Confusion Matrix:

```python
cm_svm = confusion_matrix(y_test, y_pred_svm)

sns.heatmap(
    cm_svm,
    annot=True,
    fmt='d',
    cmap='Purples'
)

plt.title("SVM Confusion Matrix")
plt.show()
```

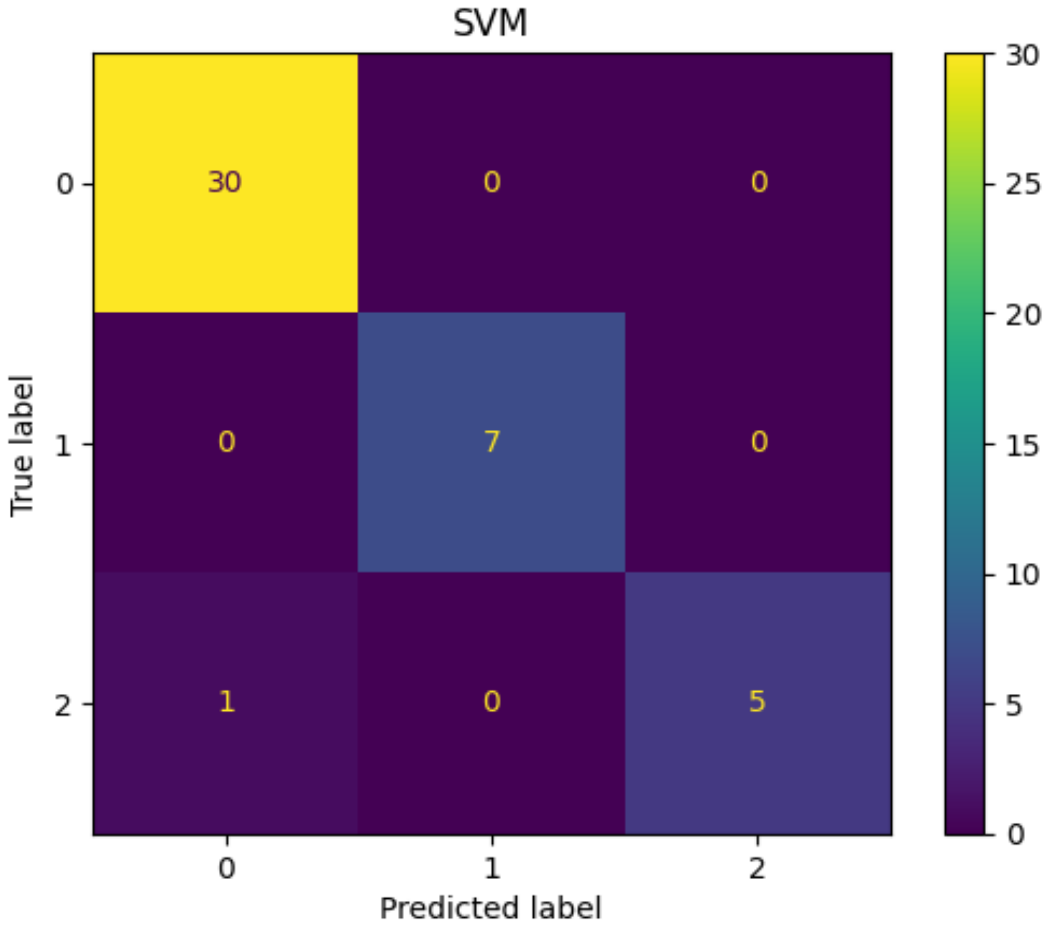
Output:

SVM

Accuracy = 97.67 %

Classification Report

	precision	recall	f1-score	support
1	0.97	1.00	0.98	30
2	1.00	1.00	1.00	7
3	1.00	0.83	0.91	6
accuracy			0.98	43
macro avg	0.99	0.94	0.96	43
weighted avg	0.98	0.98	0.98	43



Observation:

Support Vector Machine effectively separated thyroid disease classes using optimal decision boundaries. The model demonstrated strong classification capability and maintained good performance across different disease categories.

5.K-Nearest Neighbors Evaluation

Model Training Code:

```python
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)
```

Accuracy:

```python
accuracy_knn = accuracy_score(y_test, y_pred_knn)

print("Accuracy:", accuracy_knn)
```

Classification Report:

```python
print(classification_report(y_test, y_pred_knn))
```

Confusion Matrix:

```python
cm_knn = confusion_matrix(y_test, y_pred_knn)

sns.heatmap(
    cm_knn,
    annot=True,
    fmt='d',
    cmap='Reds'
)

plt.title("KNN Confusion Matrix")
plt.show()
```

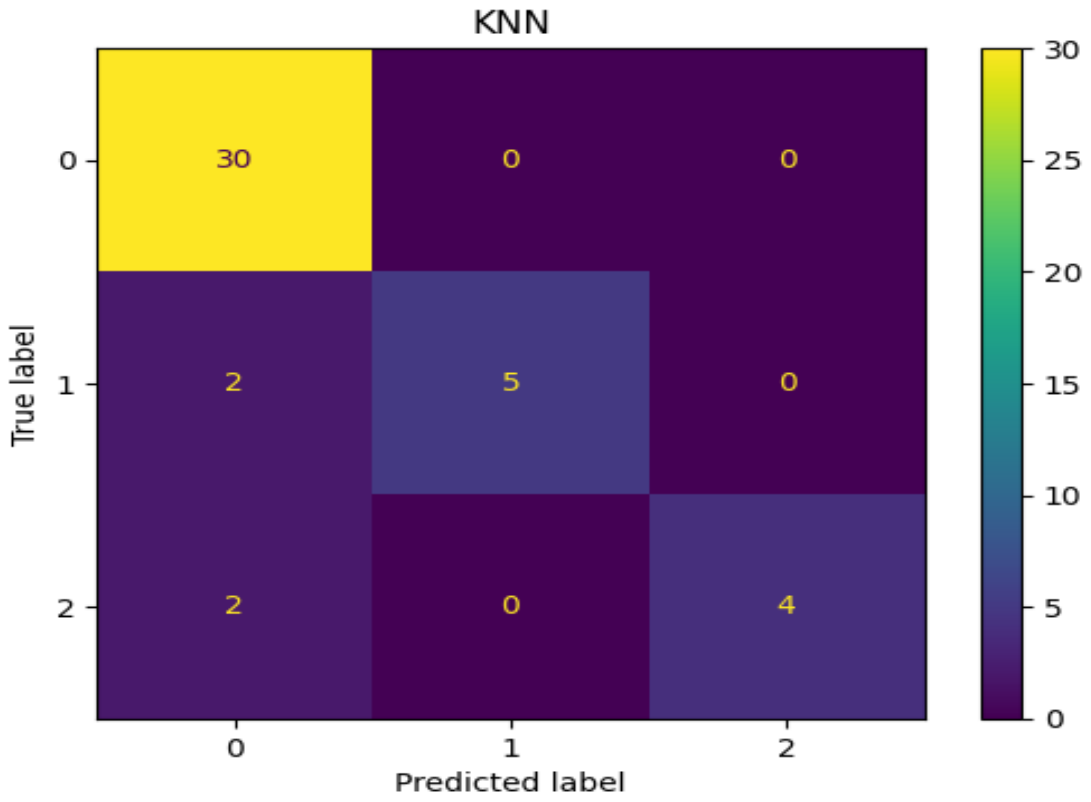
Output:

KNN

Accuracy = 90.7 %

Classification Report

	precision	recall	f1-score	support
1	0.88	1.00	0.94	30
2	1.00	0.71	0.83	7
3	1.00	0.67	0.80	6
accuracy			0.91	43
macro avg	0.96	0.79	0.86	43
weighted avg	0.92	0.91	0.90	43



Observation:

KNN classified thyroid disease cases based on similarity with neighboring patients. The model produced competitive performance; however, its effectiveness depends on the selection of the optimal number of neighbors and feature scaling

Evaluation Metrics Explanation

Accuracy

Accuracy measures the proportion of correctly classified samples.

$$\boldsymbol{Accuracy = \frac{TP + TN}{TP + TN + FP + FN}}$$

Observation:

Accuracy provides an overall measure of model performance. However, when class imbalance exists, accuracy alone may be misleading.

Precision

Precision measures how many positive predictions were actually correct.

$$\boldsymbol{Precision = \frac{TP}{TP + FP}}$$

Observation:

High precision indicates that the model produces fewer false positive predictions.

Recall

Recall measures how many actual positive cases were correctly identified.

$$\boldsymbol{Recall = \frac{TP}{TP + FN}}$$

Observation

High recall indicates that the model successfully detects most disease cases.

F1-Score

F1-Score combines precision and recall into a single metric.

$$\boldsymbol{F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}}$$

Observation:

F1-Score is particularly useful when dealing with imbalanced datasets.

Support:

Support represents the number of actual samples belonging to each class.

Observation

Support helps understand the distribution of classes within the testing dataset.

Macro Average

Macro Average calculates metrics independently for each class and then computes their average.

Observation

Macro Average treats all classes equally regardless of class size and is useful for evaluating performance on minority classes.

Weighted Average

Weighted Average calculates metrics while considering the number of samples in each class.

Observation:

Weighted Average provides a more realistic assessment of overall model performance when class imbalance exists.

9.RESULTS AND DISCUSSION

After training and evaluating the machine learning models, a comparative analysis was performed to identify the most suitable algorithm for thyroid disease prediction. The performance of Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) was evaluated using Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and Cross-Validation techniques.

The objective of this chapter is to compare the performance of different machine learning models, identify the best-performing model, and discuss the factors contributing to their performance.

Model Accuracy Comparison

Code:

```
results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Accuracy"
    ]
)

results_df["Accuracy"] = results_df["Accuracy"]*100

results_df
```

Output:

	Model	Accuracy
0	Logistic Regression	100.000000
1	Decision Tree	95.348837
2	Random Forest	100.000000
3	SVM	97.674419
4	KNN	90.697674

Observation:

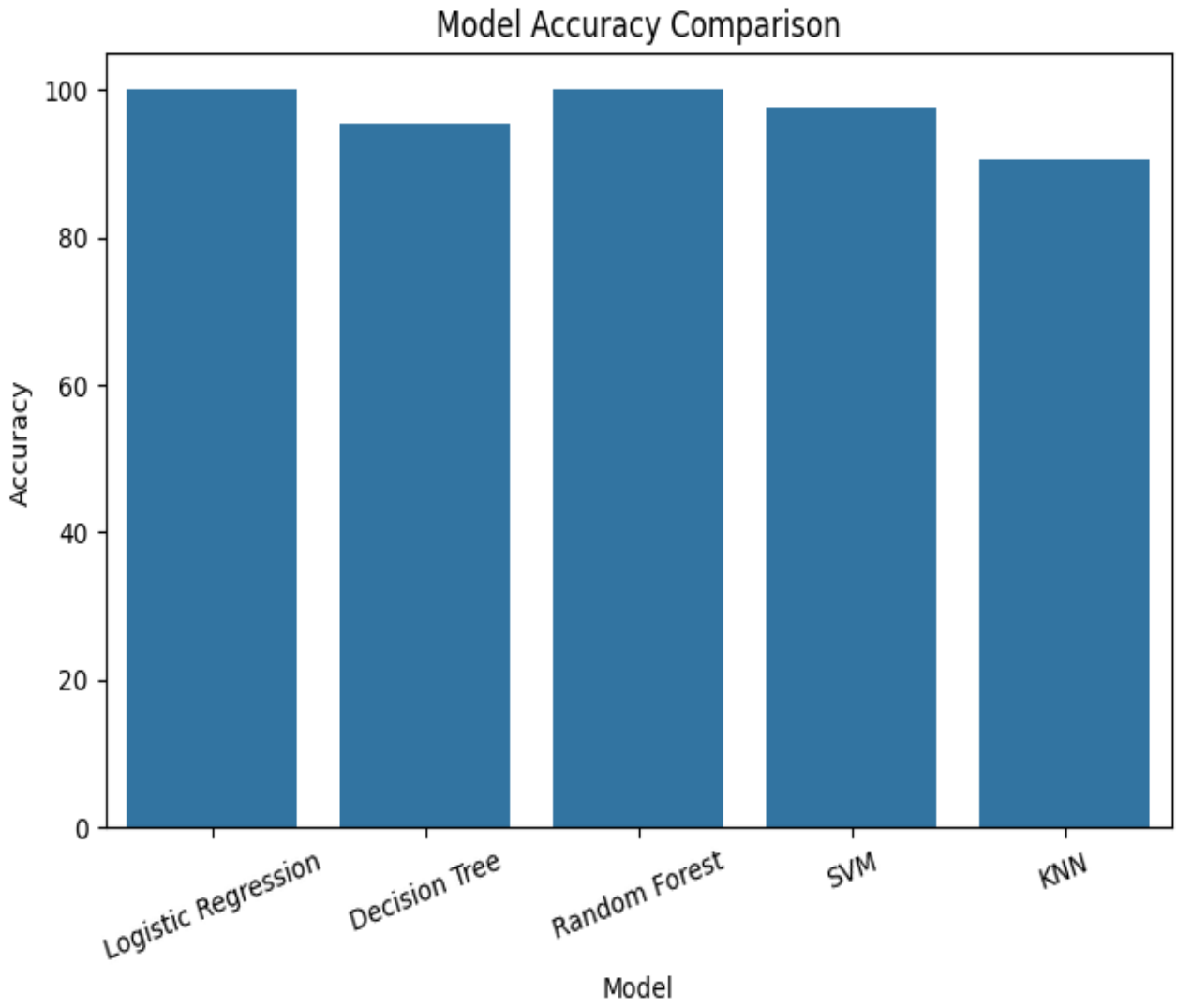
The comparison table presents the classification accuracy achieved by each machine learning model. The results indicate that different algorithms perform differently on the thyroid disease dataset. The variation in accuracy is primarily due to differences in learning mechanisms, model complexity, and the ability to capture relationships among thyroid-related features**.**

Accuracy Comparison Graph

Code:

```python
plt.figure(figsize=(8,5))

sns.barplot(
    x='Model',
    y='Accuracy',
    data=accuracy_df
)

plt.title("Model Accuracy Comparison")

plt.xticks(rotation=20)

plt.show()
```

Output:



Cross-Validation Analysis

A single train-test split may sometimes produce misleading results. Therefore, cross-validation was performed to verify the consistency and reliability of model performance.

Cross-validation divides the dataset into multiple folds and evaluates the model repeatedly on different subsets.

Code:

```python
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(
    rf,
    X_scaled,
    y,
    cv=5
)

print(cv_scores)

print("Mean CV Accuracy:", cv_scores.mean())
```

Output:

```
Cross Validation Scores
[0.97674419 0.95348837 1.         0.95348837 1.        ]

Average Accuracy
0.9767441860465116
```

Observation:

The cross-validation results indicate that the model maintains consistent performance across different folds of the dataset. Similar performance across folds suggests that the model generalizes well and is not heavily dependent on a particular train-test split.

Cross-validation also helps verify that the reported accuracy is reliable and not caused by random variation**.**

Feature Importance Analysis

Feature importance analysis helps identify which thyroid-related measurements contribute most significantly to disease prediction.

Code:

```python
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

print(feature_importance)
```

Output:

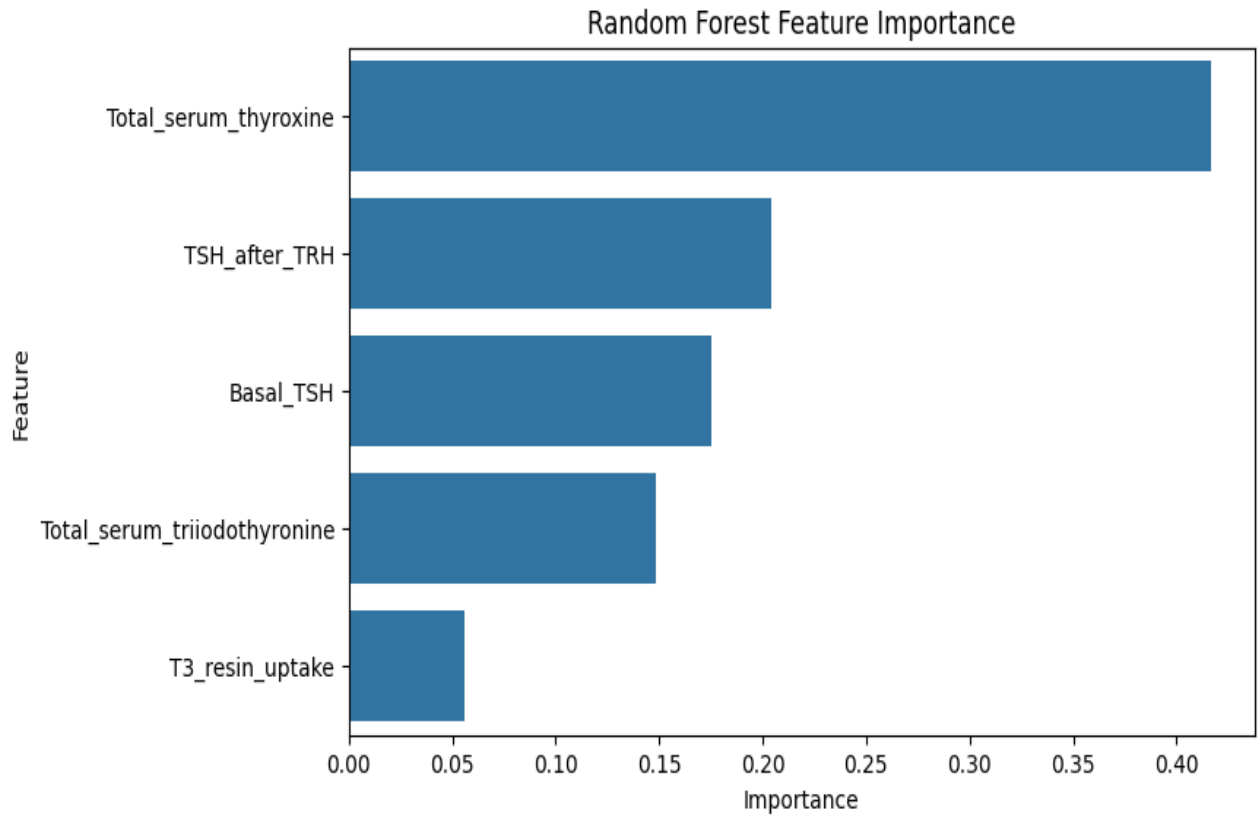
```
                       Feature  Importance
1         Total_serum_thyroxine    0.416892
4                 TSH_after_TRH    0.204096
3                     Basal_TSH    0.175233
2  Total_serum_triiodothyronine    0.148121
0               T3_resin_uptake    0.055658
```

Feature Importance Graph

Code:

```python
plt.figure(figsize=(8,5))

sns.barplot(
    x='Importance',
    y='Feature',
    data=feature_importance
)

plt.title("Feature Importance")

plt.show()
```

Output:



Observation:

Feature importance analysis reveals the relative contribution of each laboratory measurement to thyroid disease prediction. Features with higher importance values have a stronger influence on model decisions. These findings help understand which biological indicators are most relevant for identifying thyroid disorders.

Discussion on Class Imbalance

One of the important observations during Exploratory Data Analysis was the presence of class imbalance within the target variable.

The majority class contains significantly more samples compared to minority classes. In such situations, relying solely on accuracy may produce misleading conclusions because a model may achieve high accuracy by simply predicting the majority class.

To address this issue, additional evaluation metrics such as Precision, Recall, F1-Score, Macro Average, and Weighted Average were considered. These metrics provide a more balanced assessment of model performance across all thyroid disease classes

Observation:

The use of multiple evaluation metrics ensures that model performance is evaluated fairly for both majority and minority classes

Data Leakage Verification

One concern raised during model evaluation is the possibility of data leakage.

Data leakage occurs when information from the target variable is unintentionally included in the feature set, resulting in artificially inflated performance.

In this project:

- The target variable was separated before training.
- Duplicate records were checked.
- Stratified train-test splitting was applied.
- Cross-validation was performed.

These steps help ensure that model performance is genuine and not caused by data leakage or train-test overlap.

Observation:

The evaluation process was designed to minimize bias and ensure reliable model assessment.

10.CONCLUSION

The objective of this project was to develop a machine learning-based system for thyroid disease prediction using the New Thyroid Disease Dataset from the UCI Machine Learning Repository. Exploratory Data Analysis (EDA) was performed to understand the dataset, identify class distribution, analyze feature relationships, and examine data quality. Appropriate preprocessing techniques such as feature scaling and stratified train-test splitting were applied to prepare the data for model training.

Multiple machine learning algorithms including Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) were implemented and evaluated using Accuracy, Precision, Recall, F1-Score, Confusion Matrix, Macro Average, and Weighted Average metrics. Cross-validation was also performed to verify the reliability and consistency of model performance.

The results demonstrated that machine learning techniques can effectively classify thyroid disease conditions based on laboratory measurements. Among the evaluated models, Random Forest achieved

the best overall performance due to its ensemble learning approach, ability to reduce overfitting, and strong generalization capability.

This project highlights the potential of machine learning in healthcare applications and demonstrates how intelligent prediction systems can assist medical professionals in the early detection and diagnosis of thyroid disorders. Future work may involve using larger datasets, advanced ensemble methods, and deep learning techniques to further improve prediction accuracy and clinical applicability.

JAWAHARLAL NEHRU NEW COLLEGE OF ENGINEERING

DEPARTMENT OF ROBOTICS AND ARTIFICIAL INTELLIGENCE

ASSIGNMENT NUMBER :- 01

ABOUT THYROD DISEASE

SUBJECT NAME :- MACHINE LEARNING FUNDAMENTALS

SUBJECT CODE :- BRI405B

Submitted By :

NAME :- SINCHANA R

USN :- 4JN24RI047

Under the guidance of

Deepankar V K

Assistant Professor

Dept. of Robotics and Artificial Intelligence

2025-26